T.C.
IZMIR UNIVERSITY OF ECONOMICS

FACULTY OF ENGINEERING

CE 216

“SPORTS MANAGER” PROJECT

DESIGN AND ARCHITECTURE DOCUMENT

SİMGE GÖKALP - 20230602029

TUANA YAĞMUR - 20240602410

VİLDAN TURNALAR İNAL - 20253501001

UĞUR EMİN BAYNAL - 20220602408

LECTURER: KAYA OĞUZ

1. Introduction

Sports Manager application is a GUI-based management application that allows a user to manage a team within a league-based sports competition.

The application is designed as a generic sports management framework that supports multiple sports. The system architecture allows new sports to be added without modifying the core application or the user interface, ensuring extensibility through Object-Oriented Design principles.

Initially, Football will be implemented as the first sport, while the architecture supports the future addition of other sports.

2. Scope and Functionality

The system simulates a sports league and allows the user to manage a team within that league. The project includes the following main components:

League Management

- Generation of a league consisting of multiple teams
- Creation of players and coaches with randomized attributes
- Generation of a full season fixture schedule and weekly progression
- Maintenance of league standings

Team Management

The user will act as a team manager and will be able to:

- View team roster
- Select starting lineup
- Assign substitutes
- Configure team tactics
- Monitor player injuries and availability

Match Simulation

Matches will be simulated automatically by the system.
Key characteristics:

- Matches are processed in segments (halves, quarters, or sets depending on the sport)
- The user cannot directly control the players
- Tactical adjustments and substitutions can be made between segments

Injury Management

Players may become injured during matches. Injuries are tracked based on the number of matches missed, not time duration.

2.1. Functional Requirements

The functional requirements are listed below in user story format and are organized by system components, which are reflected in the requirement ID prefixes.

League Management

ID	Functional Requirement	User Story
LM 1	Start a New Game	As a player, I want to start a new game so that I can begin managing a sports team in a new season.
LM 2	Select a Sport	As a player, I want to choose the sport at the beginning of the game so that I can play the manager simulation for different sports.
LM 3	Generate a League	As a player, I want the system to generate a league with multiple teams so that I can play in a complete season competition.
LM 4	Generate Teams, Players, and Coaches	As a player, I want the system to generate teams with players and coaches so that each new game starts with a complete playable league.
LM 5	Support Sport-Specific Attributes and Positions	As a player, I want players to have sport-specific positions and attributes so that each sport feels different and realistic.
LM 6	Select a Team to Manage	As a player, I want to choose one team to manage so that I can control its lineup, tactics, and season progress. This is implied by the manager game flow described in the project brief.
LM 7	View the League Table	As a player, I want to see the league standings so that I can track my team's position during the season.
LM 8	Update League Standings After Matches	As a player, I want the league table to be updated after each match so that I can see the current rankings.
LM 9	Finish a Season	As a player, I want the system to determine whether I won the league at the end of the season so that I can see the outcome of my performance.
LM 10	Load Names and Visuals	As a player, I want team names, player names, and visual logos to be loaded from resources so that the game has a good presentation.

LM 11	Add New Sports	As a developer, I want to add a new sport without rewriting the existing UI so that the framework remains extensible.

Team Management

ID	Functional Requirement	User Story
TM 1	View the Player List	As a player, I want to see the list of players in my team so that I can make squad and tactical decisions.
TM 2	View the Team Schedule	As a player, I want to view my team's fixture schedule so that I can follow upcoming and past matches.
TM 3	Set Lineup Before a Match	As a player, I want to choose the team lineup before a match so that I can decide who will play.
TM 4	Set Tactics Before a Match	As a player, I want to set tactics before a match so that I can influence how my team performs.

Match Simulation

ID	Functional Requirement	User Story
MS 1	Simulate Matches Automatically	As a player, I want matches to be simulated by the system so that I can focus on managerial decisions rather than directly controlling players.
MS 2	Play Matches in Segments	As a player, I want matches to progress in segments such as quarters, halves, or sets so that I can intervene at break points depending on the sport.
MS 3	Change Tactics During Breaks	As a player, I want to change tactics between match segments so that I can respond to the state of the match.
MS 4	Substitute Players During Breaks	As a player, I want to substitute players during break points so that I can react to injuries, fatigue, or tactics.
MS 5	Apply Sport-Specific Scoring Rules	As a player, I want each sport to apply its own scoring and points rules so that league results are calculated correctly.

Injury Management

ID	Functional Requirement	User Story
IM 1	Track Injuries by Number of Games	As a player, I want injured players to miss a specific number of matches so that injury management affects squad selection.
IM 2	Prevent Injured Players from Being Selected	As a player, I want injured players to be unavailable for selection so that invalid match lineups cannot be created.

2.2. Non-Functional Requirements

The non-functional requirements are listed below and describe the quality attributes and architectural constraints of the system.

NFR 01 - Graphical User Interface : The system shall provide a graphical user interface that allows users to interact with the sports management simulation easily.

NFR 02 - Extensibility : The system shall support the addition of new sports without requiring modifications to the existing user interface or core logic.

NFR 03 - Maintainability : The system shall be structured using interfaces, abstract classes, and modular components to ensure maintainable and readable code.

NFR 04 - Testability : Core framework components shall be designed in a way that allows unit testing of their functionality.

NFR 05 - Reusability : Common domain entities such as Team, Player, League, and Match shall be reusable across different sports.

NFR 06 - Installability : The final application shall be installable on Windows systems.

3. System Architecture Principles

The system architecture of the Sports Manager Framework is designed according to key Object-Oriented Design principles in order to ensure extensibility and maintainability.
Since the application must support multiple sports without modifying the core system or the user interface, the architecture separates generic game logic from sport-specific behavior.

The following design principles and patterns are planned to be used in the system to implement such a system.

3.1. Interface-Based Design

The system architecture follows an interface-based design approach, where the main components interact through abstract interfaces. In this approach, the user interface communicates with the core system through generic abstractions such as Sport, League, and Player. As a result, the user interface does not depend on a specific sport implementation. Instead, it interacts with the system through common interfaces.

3.2. Factory Pattern

The Factory Pattern is used to create sport-specific objects at runtime without exposing the object creation logic to the rest of the system.
When the user starts a new game and selects a sport, the application uses a factory component to create the appropriate sport module:

➔ FootballSport

Instead of directly instantiating class in the UI, the system calls a factory method such as:

```
Sport sport = SportFactory.createSport(selectedSport);
```

This design keeps the application independent from specific sport implementations.

3.3. Layered Architecture

The architecture separates the system into two main layers as **Presentation Layer (User Interface)** and **Core Domain Layer** which is explained in the System Architecture section. This separation ensures that the user interface remains independent from sport-specific logic.

4. System Architecture

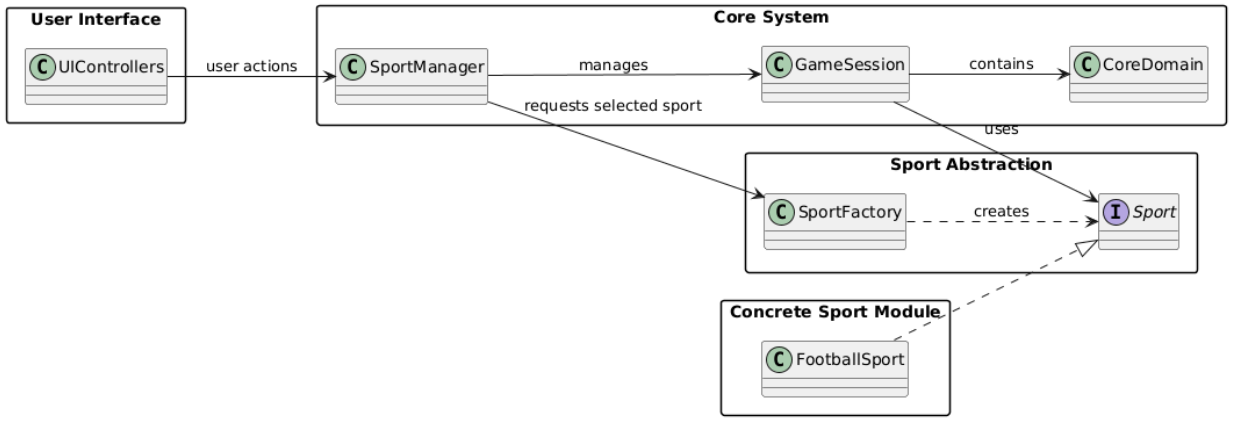
4.1 Overview

The Sports Manager application is designed based on a multi-layered structure that provides a clear separation between the UI (user interface), the core game logic, and sport-specific implementation. This system mainly aims to design a UI layer that is independent of any sport-specific classes. To achieve this, the communication between layers is grounded on abstractions. The abstraction ensures that when a new sport is added to the Sports Manager system, the existing UI design will not be affected, and it will be applicable for different sports.

In this project, the architecture has four distinct layers: User Interface, Core System, Sport Abstraction, and Concrete Sport Module. The high-level layer structure is illustrated in Figure 1, and the detailed class relationships are shown in Figure 2 (?) (Section 8.1). Between these layers, there is one-directional downward dependency, which means that upper classes are not allowed to directly import or instantiate any classes from lower layers. The only way to communicate with upper layers is via the interfaces and abstractions defined in the Sport

Abstraction layer. This limitation is the main logic of the system, which makes it extensible by design rather than by coincidence.

You can see the high level design below represented in a class diagram format.



4.2 Dependency Rule

Before describing each layer in detail, it is crucial to understand the fundamental architectural principle of the system:

- The UI layer only depends on the SportManager
- SportsManager depends on GameSession, League, Team, and the Sport interface.
- The Core System layer depends exclusively on the Sport interface and never on any concrete sport implementation such as FootballSport
- Concrete Sport Modules depend on the Sport Abstraction layer by implementing and extending its interfaces and abstract classes.

This means that when a developer adds a new sport, they only write new classes in the Concrete Sport Module. The rest of the system never needs to be changed.

4.3 User Interface Layer

The user interface layer includes JavaFX controllers that operate screen rendering and interactions with the user. The controllers in this layer are:

- MainMenuController: It displays the main menu and manages starting a new game or loading the previous one.

- SportSelectionController: It is shown at the beginning of a new game and lists available sports and sends the user's chosen sport code to SportManager.
- DashboardController: It is the main game screen, which shows weekly progression and calls SportManager.advanceWeek() to move the game forward.
- SquadController: It retrieves the player list from the management team and shows the team roster, giving the user access to player attributes and availability.
- MatchController: It controls pre-match setup (lineup, tactics) and displays match segments and outcomes.
- StandingsController: It uses SportManager.showLeagueTable() to retrieve and display the current league table.
- FixtureController: It retrieves and displays the full season fixture schedule by calling SportManager.showFixture().

Importantly, this layer does not contain game logic. All operations are delegated exclusively through SportManager. The UI controllers never reference sport-specific classes. They only interact with the Sport interface from the Sport Abstraction layer and the generic types Team, Player, Match, League, and MatchResult from the Core System layer. This implies that regardless of the sport being played, the user interface functions exactly the same.

4.4 Core System Layer

The Core System layer contains the main game management logic and all sport-independent domain entities. It is responsible for running the game loop, managing the season, and organizing all game components. The classes in this layer do not reference any concrete sport implementation.

a) SportManager

SportManager is the main entry point of the game application and the only class that the UI layer directly interacts with. It serves as a front, masking the complexity of the remainder of the system behind an easy-to-use interface. The following are the key methods of SportManager:

- startNewGame(): Initializes a new GameSession and triggers sport selection.
- selectSport(sportCode: String): Calls SportFactory.createSport(sportCode) to obtain the correct Sport implementation and stores it in the session.
- selectManagedTeam(teamId: String): Sets the team the user will manage for the season.

- advanceWeek(week: String): Progresses the game by one week; triggers training and schedules upcoming matches.
- startMatch(matchId: String): Retrieves the match for the given ID, delegates simulation to the sport-specific Match class, and returns the result to the UI.
- showLeagueTable(): Retrieves the current standings from the League object.
- showFixture(): retrieves the full fixture list from the League object.

b) GameSession

GameSession holds the complete state of the currently running game. It stores:

- currentSeasonYear: The current season number
- currentWeek: The current week within the season
- selectedSport: The active Sport implementation (stored as the interface type, never as a concrete class)
- managedTeam: The team the user is managing
- league: The active League object for the current season

It also provides isSeasonFinished() to check whether all fixtures have been played and startNextSeason() to reset the state for a new season while keeping the same sport and managed team.

c) League

League manages the full season structure. It is responsible for:

- Storing the list of all teams in the season
- Generating the full fixture schedule through generateFixtures(), which creates a round-robin schedule where each team plays every other team twice
- Retrieving all matches for a given week through getMatchesOfWeek(weekNo: int), which returns a List<Match>
- Updating the standings after a match through updateStandings(result: MatchResult), which adjusts the relevant StandingEntry objects
- Exposing the current league table through getTable(), which returns a sorted List<StandingEntry>

d) Team

Team represents a sports team and holds the following state:

- name and logoPath for display purposes
- players: List<Player>: The full squad
- coaches: List<Coach>: The coaching staff
- currentLineup: Lineup: The currently selected lineup
- currentTactic: Tactic: The currently selected tactic

e) Match

Match is an abstract class representing a match between two teams. It holds:

- weekNo, homeTeam, awayTeam: Match context
- homeScore, awayScore: running scores updated during simulation
- segments: List<MatchSegment>: The list of match segments (halves, quarters, or sets depending on the sport)
- result: MatchResult: The final outcome

It defines three abstract methods that sport-specific subclasses must implement:

- start(): Initializes the match and prepares segment structure
- play(): Simulates the next segment and updates scores and events
- finish(): Finalizes the result, determines the winner, and applies injuries

f) Player

Player is an abstract class representing a player. It holds:

- name, age, position: Identification and role
- fitness: Current physical condition, affects match performance
- injuredMatches: Number of matches the player must miss due to injury

It defines three abstract methods:

- isAvailable(): boolean: Returns whether the player can be selected for a match
- train(): Applies a training session, adjusting fitness and sport-specific attributes
- recover(): Decrements the injury counter and restores availability when the counter reaches zero

Supporting Classes

- **Coach:** holds coaching staff information: name, role, shape, training skill, and motivation skill. Coaches affect the outcome of training sessions.

- **Lineup:** holds the list of starters and substitutes for a match. It provides isValid(): boolean, which checks whether the lineup satisfies the sport-specific constraints such as minimum and maximum player counts.
- **Tactic:** holds the tactic configuration: name, shape, offense level, and defense level.
- **MatchResuls:** stores the final score, the winning team reference, and the list of InjuryRecord objects produced during the match.
- **MatchSegment:** represents a single segment of a match. It holds the segment number, a display label such as "First Half" or "Quarter 3", partial scores for home and away, and a list of event strings describing what happened during that segment.
- **InjuryRecord:** tracks a player injury. It stores the player reference, the number of games remaining, a description of the injury, and provides isRecovered(): boolean to check whether the player is ready to return.
- **StandingEntry:** tracks a single team's league record, storing played, won, drawn, lost, and points values for standings calculation and display.

4.5 Sport Abstraction Layer

The agreement that each sport must uphold is outlined in the Sport Abstraction layer. It is the line separating any sport-specific implementation from the general core system. This layer is necessary for the Core System layer. This layer is implemented by the Concrete Sport Module.

a) Sport Interface

Sport is a Java interface that declares all sport-specific behaviors. The complete set of methods is:

- getName(): String: Returns the display name of the sport, used in the UI
- createLeague(teamCount: int): League: Generates and returns a fully populated league with the given number of teams, including all teams, players, and coaches
- createTeam(name: String): Team: Creates and returns a sport-specific team instance
- createPlayer(name: String, position: String): Player: Creates and returns a sport-specific player with appropriate attributes for the given position
- createMatch(home: Team, away: Team, weekNo: int): Match: Creates and returns a sport-specific match instance

- calculatePoints(result: MatchResult, team: Team): int: Calculates and returns the points earned by the given team from the given result, applying sport-specific scoring rules
- trainTeam(team: Team): Applies a full training session to the team, updating player fitness and attributes
- applyInjuries(match: Match): Processes the match result and generates injury records for affected players

Every method in this interface is called by the Core System layer during normal game operation. Because the Core System only ever holds a reference of type Sport, it works identically for any sport that implements this interface.

b) SportFactory

SportFactory is responsible for instantiating the correct sport implementation at runtime. It exposes a single static method:

- **createSport(sportCode: String): Sport**

When the user selects a sport at the start of a new game, SportManager calls this method with the selected sport code. The factory returns the appropriate Sport implementation. Because the return type is the Sport interface, the caller never knows or needs to know which concrete class was created. Adding a new sport to the factory requires adding a single new case to this method.

4.6 Concrete Sport Module

The Concrete Sport Module contains all sport-specific implementations. For the current version of the application, this module contains the Football implementation.

- FootballSport: Implements the Sport interface and defines all Football-specific rules and configurations. It determines how Football leagues are structured, how Football players are generated with Football-specific positions such as goalkeeper, defender, midfielder, and forward, how Football matches are simulated in two halves, and how points are awarded — three for a win, one for a draw, and zero for a loss.

- FootballLeague: Extends League and adds Football-specific standing tie-breaking rules. When two teams have equal points, it compares head-to-head results, then goal difference, then a coin toss, exactly as specified in the project requirements.
- FootballMatch: Extends Match and implements start(), play(), and finish() for a two-half Football match. It handles Football-specific scoring, event generation, and injury application.
- FootballTeam: Extends Team and adds Football-specific team behavior including formation validation and Football-specific tactic constraints.
- FootballPlayer: Extends Player and adds Football-specific attributes such as shooting, passing, defending, and speed. It implements isAvailable(), train(), and recover() with Football-specific logic.

4.7 Data Flow: Starting a Game and Playing a Match

To make the architecture concrete, the following describes the complete data flow for two key scenarios.

Starting a new game:

1. The user clicks "New Game" in MainMenuController, which calls SportManager.startNewGame()
2. SportManager creates a new GameSession and navigates to SportSelectionController
3. The user selects Football; SportSelectionController calls SportManager.selectSport("football")
4. SportManager calls SportFactory.createSport("football"), which returns a FootballSport instance stored as type Sport in GameSession
5. SportManager calls sport.createLeague(20), which generates 20 Football teams with players and coaches
6. The league is stored in GameSession and DashboardController is displayed

Playing a match:

1. The user selects a match from FixtureController and confirms the lineup in MatchController
2. MatchController calls SportManager.startMatch(matchId)
3. SportManager retrieves the match from League.getMatchesOfWeek() and calls match.start()
4. For each segment, match.play() is called, generating events and updating scores

5. match.finish() is called, producing a MatchResult and triggering sport.applyInjuries(match)
6. SportManager calls league.updateStandings(result) and returns the result to MatchController
7. MatchController displays the match segments, final score, and any injuries to the user

5. Core Interfaces

This section will cover the interface structures that the application uses. The main reason for using these interface structures is that the system can run without being based on a specific sport. In other words, the application is independent of football and other sports. Instead, the general characteristics of different sports are defined in these interface structures. The classes that are created for different sports will be based on these interface structures and will include their specific characteristics and rules. The main advantage of this approach is that the system can be easily extended. If a new sport has to be included in the system in the future, there is no need to make changes to a large part of the code. Instead, new classes can be created that will be based on these interface structures. This approach is also in agreement with the Open/Closed Principle. In other words, the system is closed to changes and open to extensions.

5.1 Sport Interface

The Sport interface allows different sports to be represented in our system. Even though each sport has different rules and characteristics, they share common operations.

These operations are defined within the Sport interface. These operations are performed according to different rules for each sport. For example, our system has a FootballSport class for football. The Sport interface basically defines the following operations:

Getting the name of the sport

Creating a league

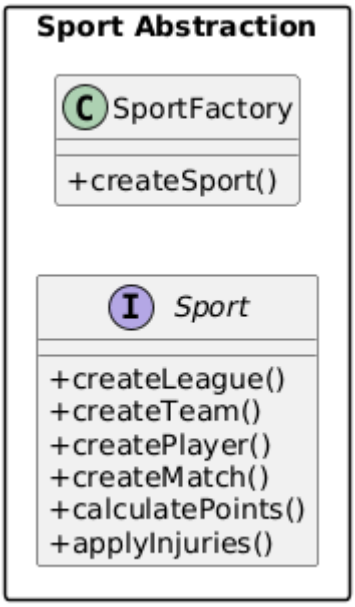
Creating a team

Creating a match

Updating points based on the match result

This structure allows the system to support different sports, and other parts of the application can function without being tied to the specifics of a particular sport.

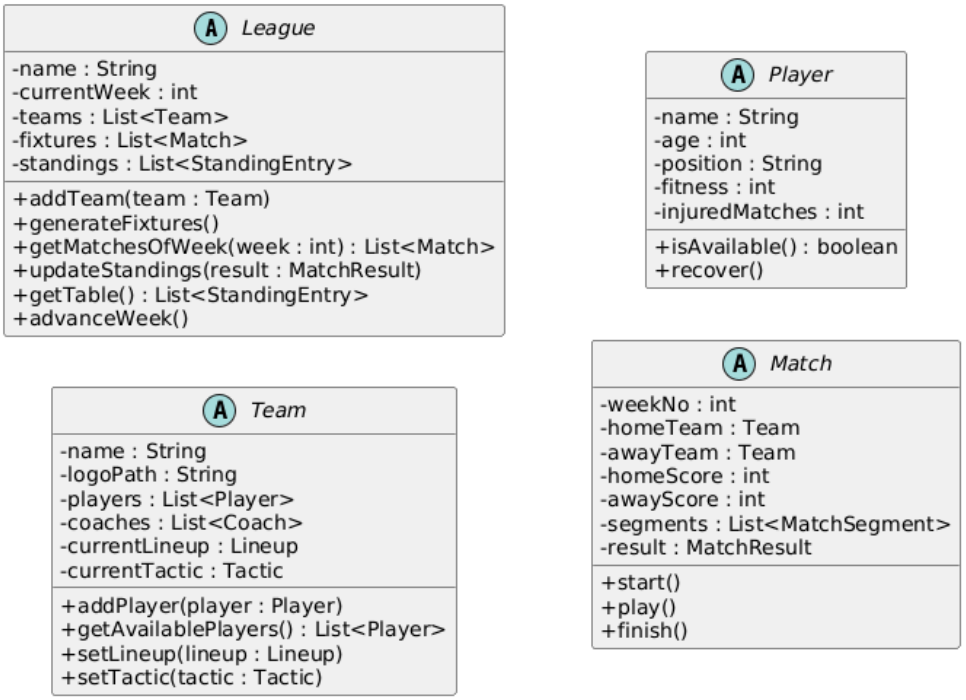
As described in the System Architecture Principles section, Factory pattern will be used to create sport specific classes.



6. Abstract Classes

Abstract classes are used to represent the basic structures that are common to many sports. The abstract classes define the features that are common to many sports.
With the use of abstract classes, the code can be written only once, as opposed to repeating the code. The specific classes for the respective sports can extend the classes to make the required modifications.
The basic abstract classes used for the project are: Player, Team, Match and League. These classes represent the core assets of the sports management system.

The class diagram for these classes are represented below.



6.1 Player Abstract Class

The Player class describes the overall structure of the players in the system. Information such as a player's name, position, skill level, and injury status is maintained in this class.

The FootballPlayer class, designed for the sport of football, can be a subclass of this class and can add additional data.

6.2 Team Abstract Class

The overall structure of a team can be defined by a class named Team. Players, coaching staff, and tactical details can be included in this class.

The FootballTeam class can be a derived class that includes specific details for football teams.

6.3 Match Abstract Class

The Match class represents a game played between two teams. Information about which teams played the game and the score is stored in this class.

The FootballMatch class, created for football, inherits from this class and implements rules specific to football matches.

6.4 League Abstract Class

The League abstract class models the core structure of a sports competition, including teams, fixtures, and league standings. It defines common league operations such as fixture

generation, match scheduling, and standings updates, while allowing sport-specific implementations to extend its behavior.

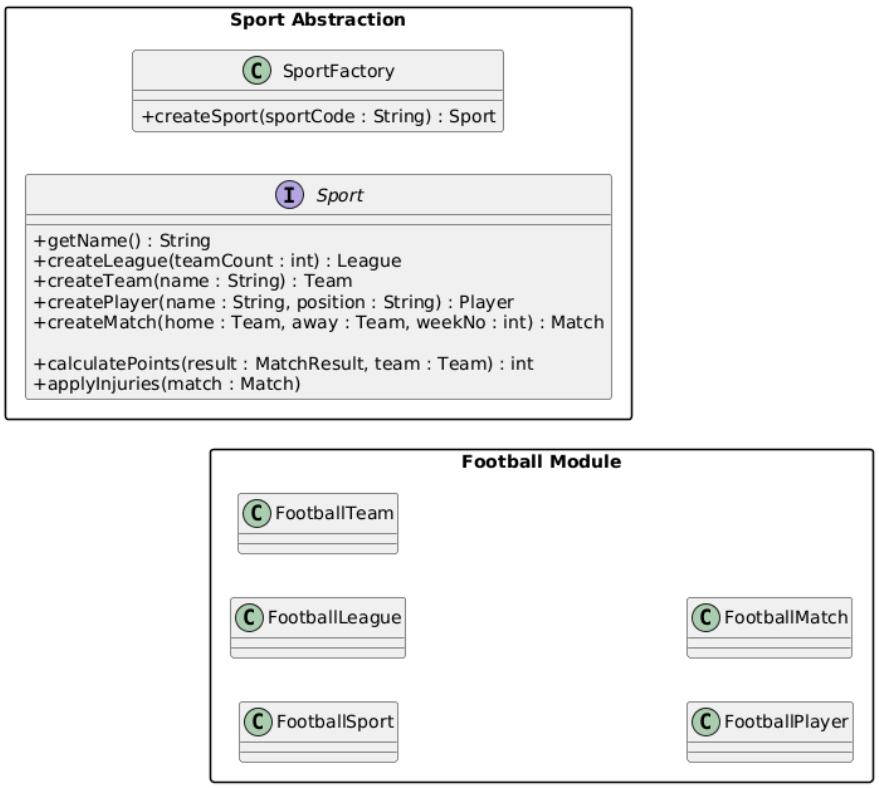
7. Sport Implementation (Football Example)

In addition, the design of the system includes abstraction and inheritance concepts for different sports. This section will discuss how the system handles the sport of football as a case study.

In the system, the sport of football is implemented in the system by using the FootballSport classes, which are derived from the interface class called Sport. Basic operations concerning the sport of football can be performed in these classes.

In addition, other classes concerning the sport of football in the system are derived from the above abstract classes using the concept of inheritance. For example, the FootballPlayer class is derived from the abstract class called Player, which contains properties concerning football players. In addition, the FootballTeam class, derived from the abstract class called Team, contains properties concerning football teams. Furthermore, the FootballMatch class, derived from the abstract class called Match, contains properties concerning football matches. Finally, the FootballLeague class, derived from the abstract class called League, contains properties concerning football leagues.

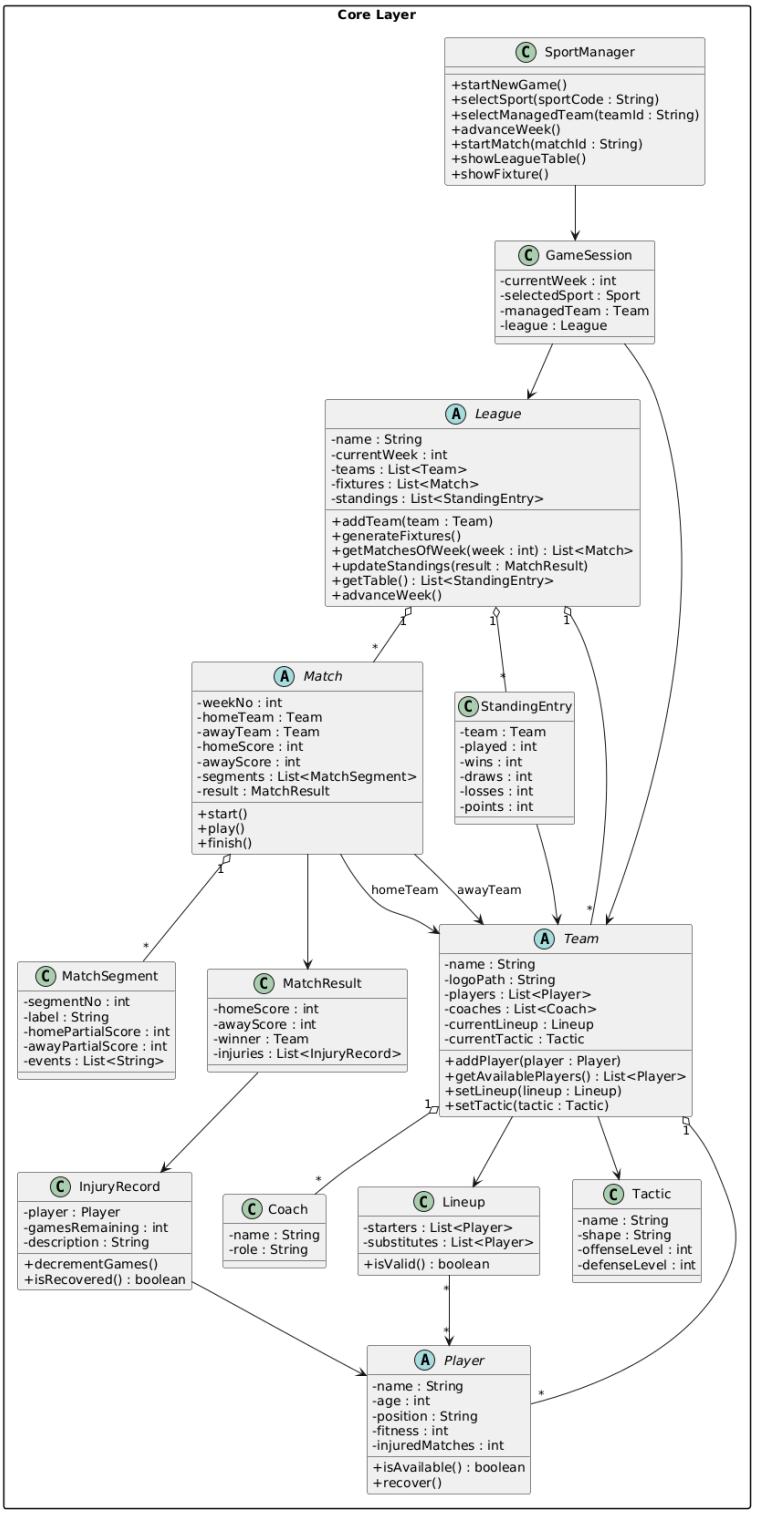
In this way, the sport of football can be easily incorporated into the system.



8. Detailed Design : Classes and Methods

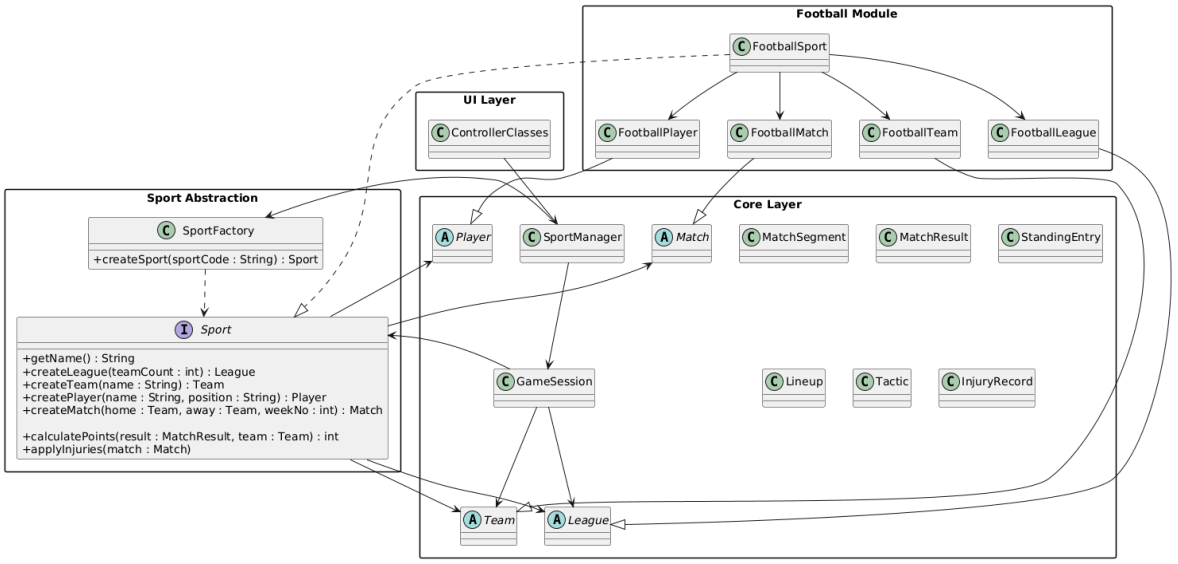
8.1. Core Layer

The Core module represents the central part of the Sports Manager framework and contains the main domain model and application logic. It includes key entities which model the structure of a sports competition. SportManager and GameSession classes coordinate the overall game flow, including league progression, match execution, and team management. This separation ensures that the core logic remains independent from the user interface and sport-specific implementations.



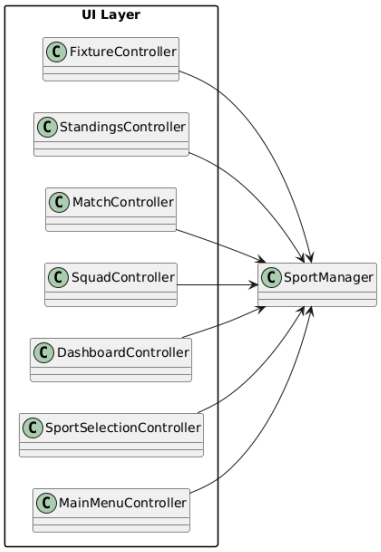
8.1. Overall Package and Class Design

The diagram below shows the full picture of classes and relations between main modules.



8.1. Draft UI Classes

UI controllers will be aligned with the graphical UI designs during the implementation phase.



9. Detailed Design : Traceability to Requirements

The requirements are mapped to the corresponding classes and methods to ensure traceability between the system requirements and the detailed design. The mappings are presented in the tables below.

League Management Mapping

ID	Functional Requirement	Primary Class	Related Method(s)	Explanation
LM-1	Start a New Game	SportManager	startNewGame()	Starts a new game session and initializes the simulation state.
LM-2	Select a Sport	SportManager, SportFactory	selectSport(), createSport()	Creates the selected sport implementation through the factory.
LM-3	Generate a League	Sport, League	createLeague(), addTeam(), generateFixtures()	Creates the league structure and generates the fixture schedule.
LM-4	Generate Teams, Players, and Coaches	Sport, Team	createTeam(), createPlayer(), addPlayer()	Creates teams and populates them with players and coaches.
LM-5	Support Sport-Specific Attributes	Sport, Player	createPlayer(name, position)	Creates sport-specific player instances with appropriate attributes.
LM-6	Select a Team to Manage	SportManager	selectManagedTeam(teamId)	Assigns the selected team to the active game session.
LM-7	View the League Table	SportManager, League	showLeagueTable(), getTable()	Displays the current league standings.
LM-8	Update League Standings	League, Sport	updateStandings(result), calculatePoints()	Updates standings and points after each match.
LM-9	Finish a Season	League, GameSession	advanceWeek(), getMatchesOfWeek()	Season completion is inferred when all fixtures are played.
LM-10	Load Names and Visuals	—	—	Not represented directly in the UML; can be implemented with a ResourceLoader.
LM-11	Add New Sports	SportFactory	createSport(sportCode)	Allows the addition of new sport modules without modifying the UI.

Team Management Mapping

ID	Functional Requirement	Primary Class	Related Method(s)	Explanation
TM-1	View the Player List	Team	getAvailablePlayers()	Returns the list of players currently available for selection.
TM-2	View the Team Schedule	SportManager, League	showFixture(), getMatchesOfWeek()	Displays the team's upcoming and past matches.
TM-3	Set Lineup Before a Match	Team, Lineup	setLineup(), isValid()	Defines and validates the starting lineup and substitutes.
TM-4	Set Tactics Before a Match	Team	setTactic(tactic)	Allows the manager to configure tactical parameters.

Match Simulation Mapping

ID	Functional Requirement	Primary Class	Related Method(s)	Explanation
MS-1	Simulate Matches Automatically	SportManager, Match	startMatch(), start(), play(), finish()	Runs the automated match simulation.
MS-2	Play Matches in Segments	Match, MatchSegment	play()	Processes the match flow through multiple segments.
MS-3	Change Tactics During Breaks	Team	setTactic()	Allows tactical adjustments during match breaks.
MS-4	Substitute Players During Breaks	Team, Lineup	setLineup()	Allows substitutions during match intervals.
MS-5	Apply Sport-Specific Scoring Rules	Sport	calculatePoints()	Applies sport-specific scoring logic.

Injury Management Mapping

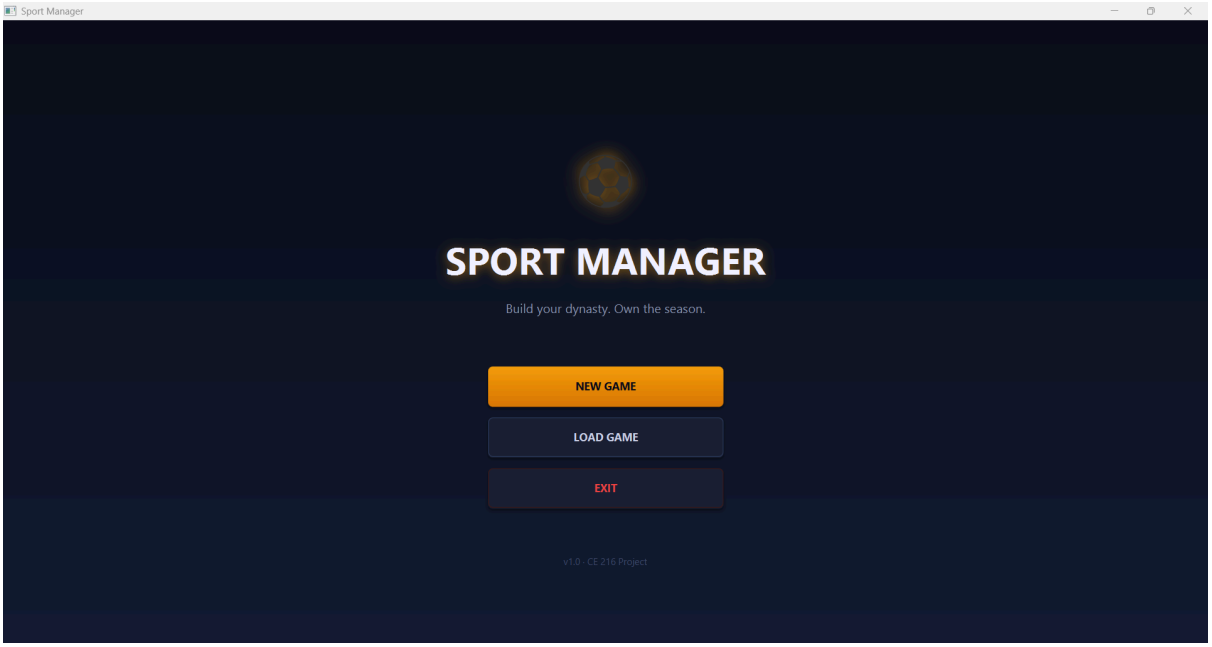
ID	Functional Requirement	Primary Class	Related Method(s)	Explanation
IM-1	Track Injuries by Number of Games	Sport, InjuryRecord, Player	applyInjuries(), decrementGames(), recover()	Tracks injuries based on the number of matches missed.
IM-2	Prevent Injured Players from Being Selected	Player, Team	isAvailable(), getAvailablePlayers()	Ensures injured players cannot be selected for matches.

10. Detailed Design: User Interfaces

The Sports Manager application provides a graphical user interface built with JavaFX, following a dark sports-management aesthetic with a deep navy palette and amber accent colours. All screens share a consistent visual language: a top navigation bar, card-based content panels, and a unified CSS theme applied globally through style.css. The UI is strictly read-only with respect to the domain — every screen only calls methods on SportManager and renders what is returned through abstract types. No controller is aware of which concrete sport is loaded.

10.1 Main Menu Screen

Controller: MainMenuController | FXML: main-menu.fxml



The Main Menu is the application's entry point. It is displayed immediately when the application launches. The screen features a full-height centred layout with a large football emoji icon, the application title "SPORT MANAGER", and a subtitle line. Below these, three vertically stacked action buttons are presented:

New Game — starts a new game session by calling SportManager.startNewGame(), which resets the GameSession and navigates to the Sport Selection screen.

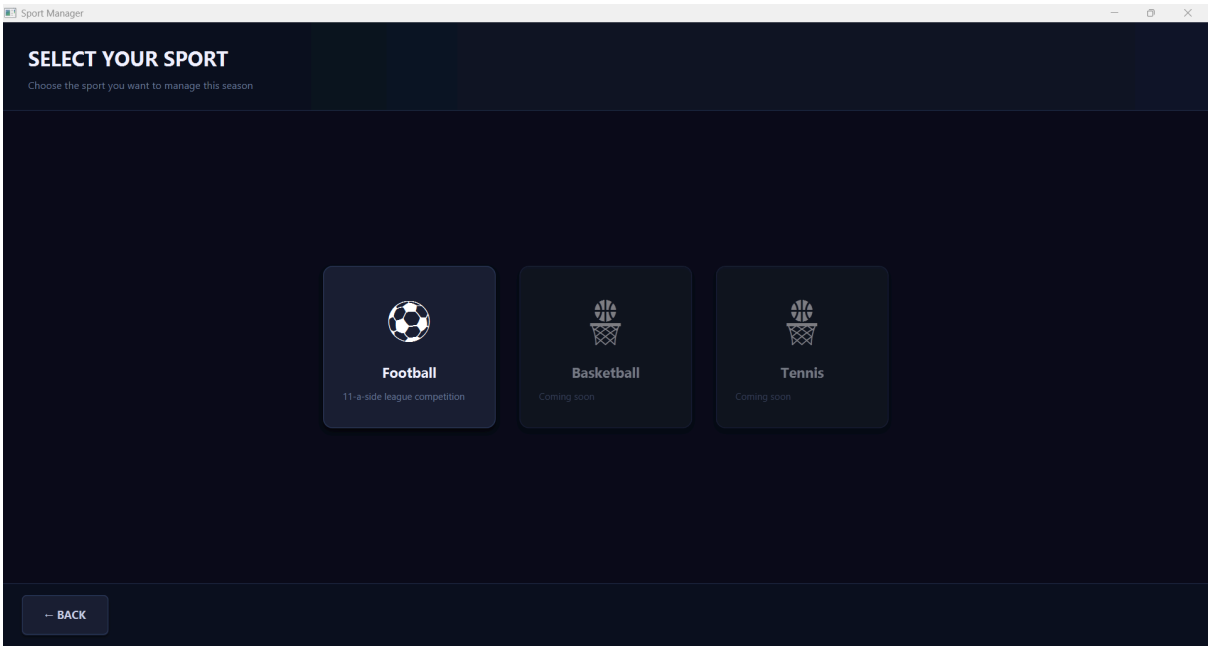
Load Game — reserved for future functionality; currently displays an informational dialog.

Exit — closes the application.

The design purpose of this screen is minimal: it provides a clean, branded starting point and delegates all responsibility immediately to SportManager.

10.2 Sport Selection Screen

Controller: SportSelectionController | FXML: sport-selection.fxml



The Sport Selection screen is shown after the user starts a new game. Its purpose is to allow the user to choose which sport they wish to manage for the season (Requirement LM-2). The screen header displays the title "SELECT YOUR SPORT". The body contains a scrollable FlowPane populated dynamically by the controller at runtime — one card is generated per available sport.

Each sport card contains:

A large sport emoji icon

The sport's display name in bold

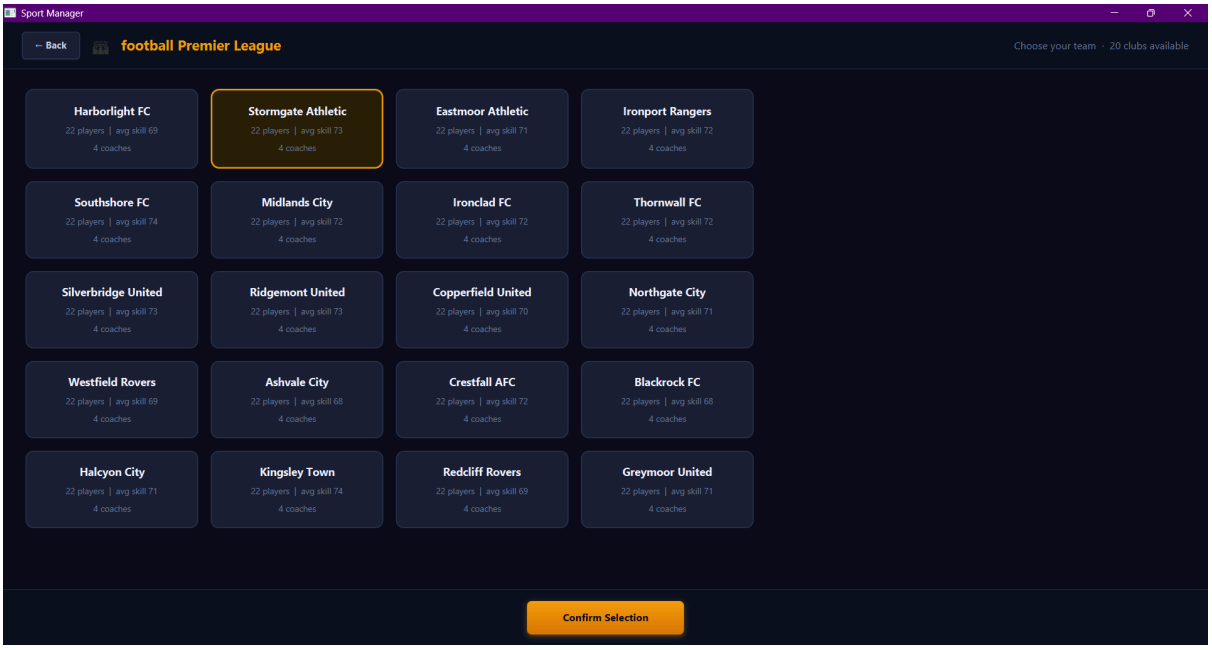
A short description line beneath it

Currently, Football is the only enabled sport, styled with full opacity and a hover glow effect. Future sports (Basketball, Tennis) are displayed at reduced opacity with the cursor blocked, visually communicating that they are planned but not yet implemented. When the user clicks an enabled card, SportManager.selectSport(code) is called, which triggers league generation and navigates forward.

A Back button in the bottom bar returns the user to the Main Menu.

10.3 Team Selection Screen

Controller: TeamSelectionController | FXML: team-selection.fxml



The Team Selection screen allows the user to choose which team they will manage for the season (Requirement LM-6). The top bar displays the generated league name and a subtitle showing how many clubs are available.

The centre of the screen contains a scrollable GridPane arranged in four columns. Each cell contains a team card displaying:

The team's name in bold

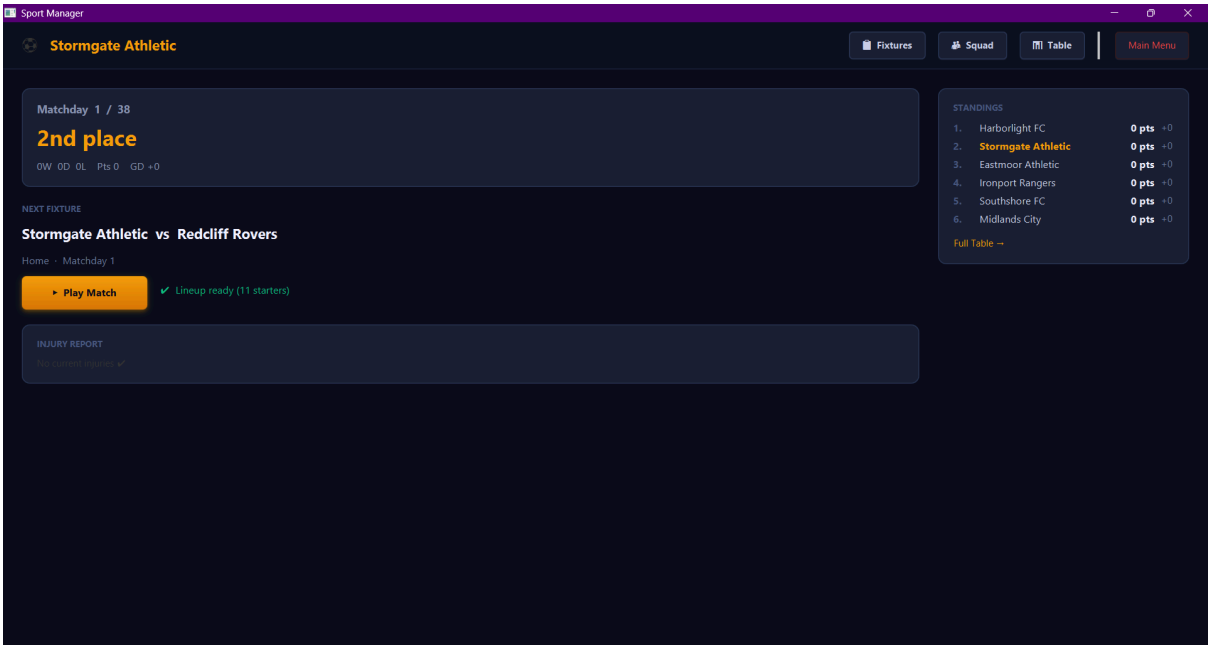
The number of players in the roster and their average skill score

The number of coaching staff members

Teams are generated by FootballSport.createLeague(20) before this screen is shown, so the user sees all 20 randomly named clubs with realistic roster statistics. Clicking a card selects it, which highlights its border in amber. Clicking a different card deselects the previous one. The user confirms their choice with the Confirm Selection button at the bottom, which calls SportManager.selectManagedTeam(team) and navigates to the Dashboard.

10.4 Dashboard Screen

Controller: DashboardController | FXML: dashboard.fxml



The Dashboard is the central hub of the game and the screen the user returns to after every match (Requirement TM-2, LM-7, LM-9). It is divided into a top navigation bar, a left content column, and a right standings sidebar.

Top navigation bar displays the managed team's name in amber and provides four navigation buttons:

Fixtures → navigates to the Fixture screen

Squad → navigates to the Squad screen

Table → navigates to the Standings screen

Main Menu → returns to main menu after a confirmation dialog

Left column contains three stacked cards:

Week/Season card — displays the current matchday number (e.g. "Matchday 5 / 38"), the team's current league position (e.g. "3rd place"), and the season record (W/D/L, points, goal difference).

Next Fixture card — shows the upcoming match (e.g. "Ironclad FC vs Silverbridge United"), the venue (Home/Away) and matchday number. The Play Match button triggers SportManager.startMatch(). A line beneath the button shows the lineup validity status — green if the starting XI is complete, amber warning if the lineup needs attention.

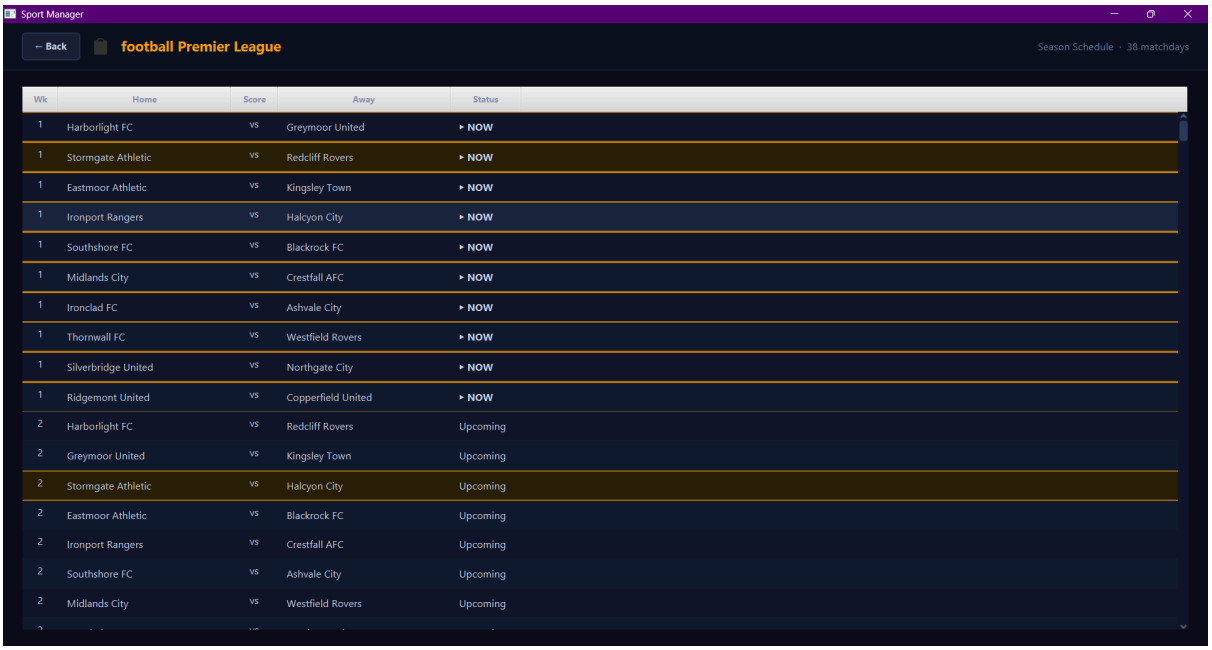
Injury Report card — lists every currently injured player in the squad with their position and the number of matches remaining in their recovery. If no players are injured, a green confirmation message is shown.

Right sidebar contains a mini standings table showing the top six clubs with position, name, points, and goal difference. If the managed team sits below sixth place, their row is appended below a continuation ellipsis. The managed team's row is always highlighted in amber.

When the season is complete (isSeasonFinished() returns true), the Dashboard replaces the fixture card with a season-over banner displaying the champion's name.

10.5 Fixture Screen

Controller: FixtureController | FXML: fixture.fxml



The Fixture screen displays the full season schedule across all matchdays (Requirement TM-2). It is reached from the Dashboard's Fixtures button and calls SportManager.showFixtureData() to retrieve all rounds.

The screen body contains a single TableView with five columns: Wk (matchday number), Home, Score, Away, and Status. On load, the view automatically scrolls to the current matchday so the user does not need to search manually.

Each row is styled based on its status:

Played rows show the final score in the Score column and use muted grey text to indicate completed fixtures.

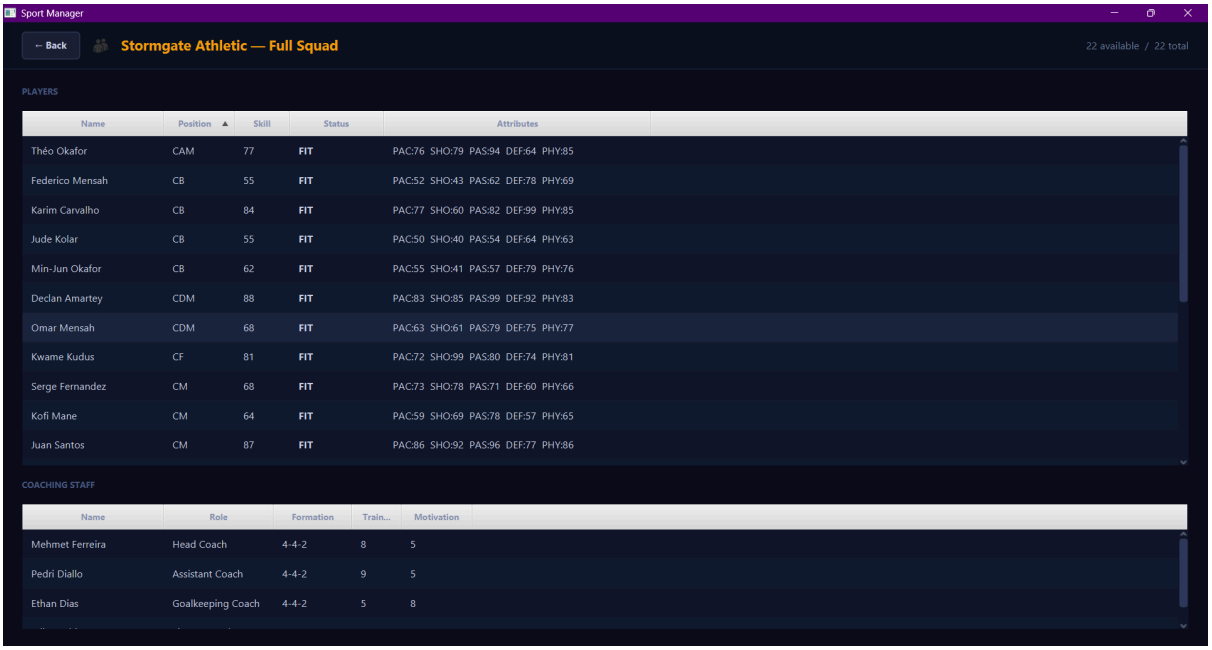
Current matchday rows are highlighted with an amber "▶ NOW" badge in the Status column.

Upcoming rows show "vs" in the Score column.

All rows where the managed team is involved (home or away) are highlighted with an amber background tint, making it easy to track the team's schedule within the full fixture list. A Back button in the top bar returns to the Dashboard.

10.6 Squad Screen

Controller: SquadController | FXML: squad.fxml



The Squad screen provides a complete view of the managed team's roster and coaching staff (Requirements TM-1, LM-4, IM-2). It is reached from the Dashboard's Squad button.

The top bar displays the team name and an availability count (e.g. "19 available / 22 total").

The Players table occupies the majority of the screen and contains five columns:

Name — the player's full name

Position — abbreviated position code (GK, CB, CM, ST, etc.)

Skill — the player's overall skill rating (55–89)

Status — "FIT" in green or "INJ (N)" in red, where N is the number of matches remaining

Attributes — a compact summary of the player's sport-specific ratings (e.g. Pace:78 Shooting:72 Passing:65)

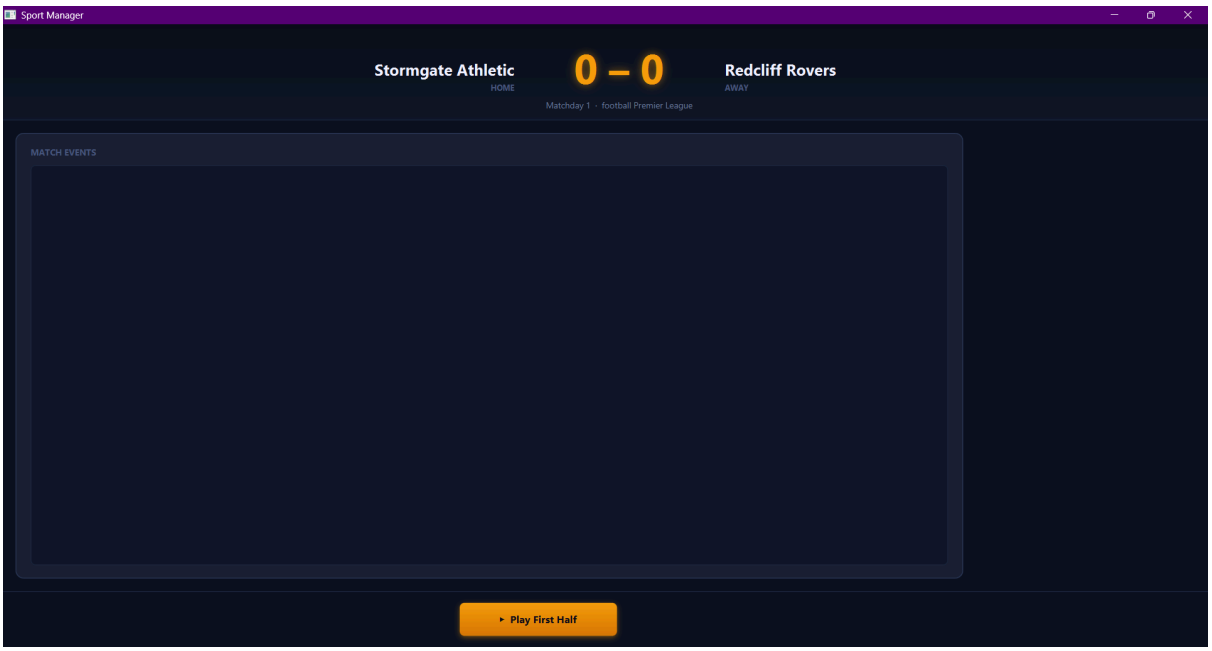
Injured players have their entire row shaded in red, giving an immediate visual indication of squad availability. The Status cell's text colour reinforces this: green for available, red for injured.

The Coaching Staff table below the player table lists all coaches with their name, role (Head Coach, Assistant, GK Coach, Fitness Coach), preferred formation, training skill rating, and motivation skill rating.

A Back button returns to the Dashboard.

10.7 Match Screen

Controller: MatchScreenController | FXML: match-screen.fxml



The Match Screen handles the full match experience: it shows the live score, renders match events segment by segment, and provides the half-time intervention panel (Requirements MS-1 through MS-5).

Scoreboard panel at the top displays the home team name on the left, the away team name on the right, and the current score in large amber digits in the centre. A matchday and league label appears below the score.

Event log occupies the left portion of the centre area inside a scrollable card. Events are added incrementally as each segment is simulated. Each event type is colour-coded:

Goal events Injury events

Yellow card events

Substitution events

Tactic change events

A segment header (e.g. "FIRST HALF") and a segment score summary (e.g. "End of First Half: 1 – 0") are inserted between segments.

Intervention panel appears on the right side at half time and contains:

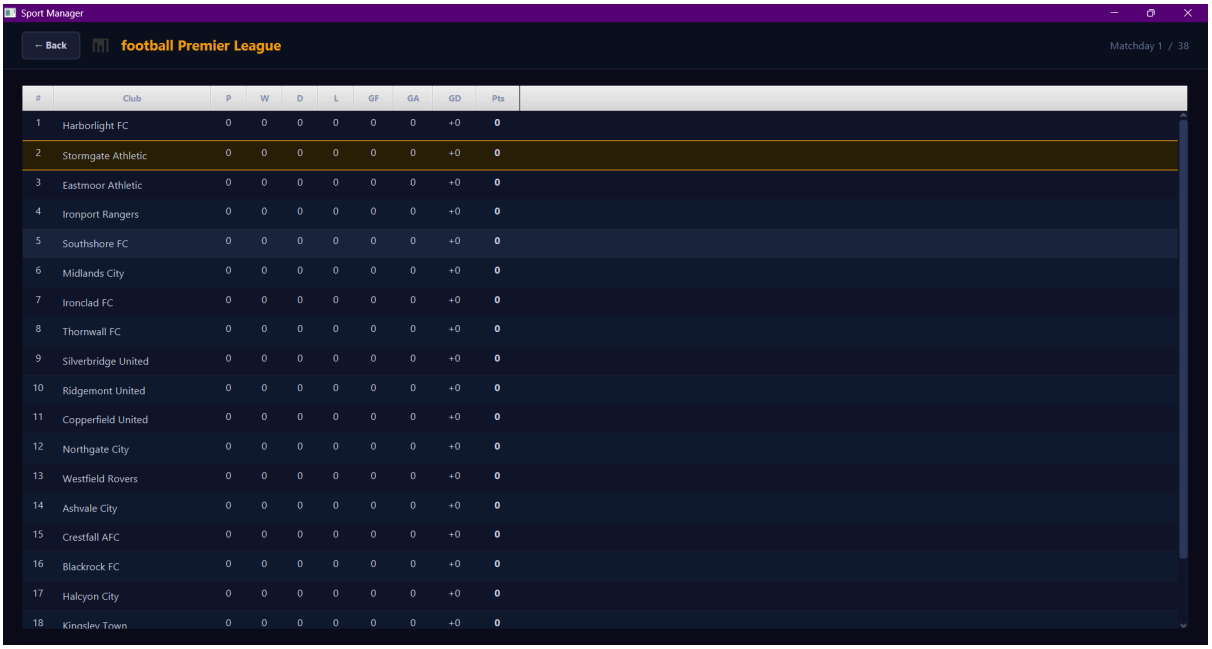
A substitution section with two dropdowns (remove player / add from bench) and a counter tracking substitutions used out of a maximum of three

A tactic section with a dropdown showing all available formations, and an "Apply Tactic" button

The bottom bar holds the simulation button, whose label updates to reflect the next segment (e.g. "▶ Play First Half" → "▶ Play Second Half"). After the match ends, the simulation button is replaced by a Continue button which calls SportManager.advanceWeek(), finalising the round and returning to the Dashboard.

10.8 League Table Screen

Controller: LeagueTableController | FXML: league-table.fxml



The League Table screen shows the full sorted standings for the current season (Requirement LM-7). It is accessible from the Dashboard's Table button and calls SportManager.showLeagueTableData() which returns a List<StandingEntry> already sorted by points → goal difference → goals for.

The screen body contains a TableView with ten columns: # (position), Club, P (played), W, D, L, GF, GA, GD, and Pts. All numeric columns are centre-aligned; the Pts column is bold. The managed team's row is highlighted in amber to allow the user to immediately identify their standing among all twenty clubs.

The top bar shows the league name and the current matchday. A Back button returns to the Dashboard.

10.9 Visual Design Principles

All screens adhere to the following shared design principles:

Colour palette: Deep navy background (#0a0e1a), card surfaces (#1c2235), amber accent (#f59e0b), green success (#10b981), red danger (#ef4444).

Typography: Bold uppercase labels for section headers with increased letter-spacing; regular weight for data rows.

Consistent navigation: Every secondary screen has a "← Back" button in the top-left corner returning to the Dashboard, creating a predictable navigation model.

State feedback: Injured rows, managed-team rows, current-week rows, and played fixtures each have distinct visual treatments so the user can read game state at a glance without requiring tooltips or popups.

No sport-specific visuals in shared screens: The League Table, Fixture screen, and Squad screen render correctly for any sport because they only read from the abstract StandingEntry, Match, and Player types — consistent with the UI independence requirement (NFR-02).

11. Extensibility

11.1 Design Principle

The Sports Manager framework is built so that new sports can be added without modifying any existing code in the UI or Core System layers. This follows the Open/Closed principle which means the system is open for extension but closed for modification. Because no class in the Core System or UI holds a reference to any concrete sport class, there is no existing code to change when a new sport is introduced.

11.2 How to Add a New Sport

Adding a new sport requires only four steps:

1. **Implement the Sport interface:** Create a class such as VolleyballSport that defines all sport-specific behavior: player generation, match simulation, points calculation, and injury logic.
2. **Extend the abstract classes:** Create VolleyballMatch, VolleyballPlayer, VolleyballTeam, and VolleyballLeague by extending the corresponding abstract classes and adding sport-specific attributes and rules.
3. **Register in SportFactory:** Add a single new case to SportFactory.createSport() so the factory returns the correct implementation when the user selects the sport.
4. **Add to the selection screen:** Add the sport name to the list in SportSelectionController.

11.3 What Remains Unchanged

When a new sport is added, every existing class remains untouched—all UI controllers, SportManager, GameSession, League, Match, Team, Player, and all game logic. The only existing file modified is SportFactory, where one line is added. This confirms that the architecture fully satisfies the project requirement that the UI relies on an interface rather than a real implementation.

12. Technology Stack

The following technologies will be used:

- Language : Java
- GUI Framework : JavaFX
- Testing Framework : JUnit 5
- Build System : Maven
- Deployment : jpackage to generate a Windows installer
- Version Control : Git